OFFICIAL MASTER TECHNICAL DEFENSE & VIVA REFERENCE

NOUS AI

Enterprise AI Resume Intelligence, Multi-Model Role Scoring & Real-Time Direct Career Portal Crawling Engine

1. RESILIENT INGESTION & SECURITY

Apache Tika magic byte sniffing, ClamAV anti-malware TCP daemon, and SHA-256 deduplication.

2. PROJECT-HEAVY AI RECRUITER

45% project architecture weighting; 3-tier cascade across Gemini Live, OpenAI, and local parser.

3. DIRECT ENTERPRISE CRAWLING

Real-time ATS adapters (Greenhouse, Lever, Amazon, Uber) with soft-expiration and daily cron.

4. REACTIVE STREAMING UX

Server-Sent Events (SSE) push streaming with 1500ms REST polling fallback in React 19.

Backend: Java 17, Spring Boot 3.3.4, PostgreSQL (Neon DB)

Frontend: React 19, Vite, Vanilla CSS

Purpose: Comprehensive Technical Viva, Architecture & Resume Defense

1. Executive Summary & Spoken Elevator Pitch

1.1 The 30-Second Elevator Pitch (Word-for-Word for Interviews)

> *"Nous AI is an enterprise-grade resume intelligence platform and direct job discovery engine. Unlike traditional job boards that rely on shallow keyword matching and stale aggregators, Nous AI evaluates candidates like a Staff Recruiter—giving 45% weight to verified project architectures. It features a 3-tier zero-failure AI fallback cascade, crawls live corporate ATS portals directly (Amazon, Stripe, Uber) with SHA-256 deduplication, and streams real-time evaluation updates to a React 19 single-page dashboard using Server-Sent Events with an automatic polling fallback."*

1.2 Complete Technology Stack Matrix & Technical Rationale

Layer	Technology	Exact Version	Architectural Responsibility & Interview Rationale
Frontend UI	React + Vite	React 19.0.0, Vite 5.x	Fast SPA; sub-50ms client-side DOM rendering with zero server compilation overhead.
Styling	Vanilla CSS Tokens	CSS3 Custom Props	Custom dark theme (`#0f172a`, `#1e293b`), glassmorphism, responsive cards, zero CSS bundle bloat.
Backend Core	Java 17 + Spring Boot	Spring Boot 3.3.4	Dependency injection, thread pool management, non-blocking `@Async`, RFC 7807 problem details.
Concurrency Pool	ThreadPoolTaskExecutor	Spring Core Async	`scanTaskExecutor` (Core: 4, Max: 10, Queue: 50) immediately frees Tomcat HTTP threads.
Real-Time Streaming	Server-Sent Events (SSE)	Spring SseEmitter	Unidirectional event streaming (`text/event-stream`) over HTTP without WebSocket overhead.
Persistence	PostgreSQL (Neon DB)	PostgreSQL 16	Relational consistency, cascade deletions, unique SHA-256 hash indexes, serverless cloud hosting.
Connection Pool	HikariCP	5.1.0	Tuned for cloud containers (`maxPoolSize=5`, `minIdle=2`, `idleTimeout=300000ms`, `prepareThreshold=0`).
MIME Security	Apache Tika	2.9.2	Sniffs true magic bytes in file headers, blocking spoofed `.exe` files renamed as `.pdf`.
Text Extractors	Apache PDFBox & POI	PDFBox 3.0.3, POI 5.3.0	PDFBox extracts text with reading-order sorting; POI extracts XML text from `.docx` tables/paragraphs.
Anti-Malware	ClamAV Socket Client	TCP Socket :3310	Streams raw file bytes to ClamAV daemon prior to disk persistence.
AI Engine	Google Gemini & OpenAI	gemini-flash-latest / gpt-4o-mini	Project-weighted role prediction, strict JSON schema output, and 6-domain semantic fallback.
Web Scraping	Jsoup & RestTemplate	Jsoup 1.17.2	Directly queries official public ATS APIs (Greenhouse, Lever, Amazon) and Schema.org JSON-LD.

2. Resume Word-by-Word Defense Guide (Part 1)

This section unpacks the exact terminology written on your resume so you can explain every single word with complete technical depth:

📌 Resume Bullet Point 1:

> *"Built a full-stack AI resume platform using Spring Boot and React, delivering a smooth, real-time user experience via Server-Sent Events (SSE) with a reliable automatic polling fallback."*

Term on Resume	Simple Meaning	Exact Code Implementation in Nous AI	What to Say in the Interview
Full-Stack	Both Frontend UI and Backend Server/Database.	React 19 Frontend + Spring Boot 3.3.4 Backend + PostgreSQL (Neon DB).	*"I engineered the full application lifecycle—from React state hooks to Spring Boot controllers, async workers, and database schemas."*
Spring Boot	Enterprise Java framework for backend REST APIs.	Spring Boot 3.3.4 managing REST controllers, JPA persistence, thread pools, and RFC 7807 exceptions.	*"Spring Boot gave us production-grade dependency injection, asynchronous thread pool management, and robust error handling."*
React	Interactive JavaScript library for building single-page UIs.	React 19 with Vite, custom hooks (`useScanStatus`), filter sliders, and sub-50ms DOM updates with zero reloads.	*"React allowed us to build a reactive single-page dashboard with instant filter controls and real-time state transitions."*
Real-Time UX	Live progress updates without clicking refresh.	Progress stepper automatically advances (`PENDING` ➔ `PROCESSING` ➔ `COMPLETE`) on screen.	*"The UI reflects backend pipeline advancements instantly without freezing the browser or requiring manual refreshes."*
Server-Sent Events (SSE)	Standard HTTP protocol for server-to-client unidirectional push.	Spring `SseEmitter` streaming `text/event-stream` JSON packets over `/api/scans/:id/events` to browser `EventSource`.	*"SSE provides lightweight push streaming over standard HTTP with native browser reconnection and zero WebSocket protocol overhead."*
Automatic Polling Fallback	Backup request mechanism if live stream is blocked.	`useScanStatus.js` catches SSE network errors and automatically polls `GET /api/scans/:id` every 1500ms.	*"If a corporate proxy or firewall blocks SSE streams, the React hook gracefully downgrades to 1500ms REST polling so the app never gets stuck."*

📌 Resume Bullet Point 2:

> *"Designed a smart, LLM-powered role classifier using Google Gemini and a custom semantic fallback parser to accurately extract candidate skills and match them to ideal roles."*

Term on Resume	Simple Meaning	Exact Code Implementation in Nous AI	What to Say in the Interview
LLM-Powered	Driven by Large Language Models understanding natural language.	Integrates Google Gemini Live REST API (`gemini-flash-latest`) and OpenAI `/chat/completions`.	*"We leverage generative LLMs to evaluate resume context, project descriptions, and technical architectures rather than dumb keyword counting."*
Role Classifier	AI module predicting top 3 matching job titles.	Outputs Rank 1, 2, 3 roles with confidence scores (0.65–0.96), match reasons, and skill arrays.	*"The classifier acts as a Staff Recruiter, ranking the top 3 best-fitting corporate engineering roles with calibrated match percentages."*
Google Gemini	Google's fast generative AI model family.	Direct REST calls with model cascade (`gemini-flash-latest` ➔ `2.5-flash` ➔ `2.0-flash` ➔ `1.5-flash`).	*"We use Gemini Flash for sub-2-second structured JSON role evaluation with custom system prompt enforcement."*
Custom Semantic Fallback Parser	Built-in local engine analyzing skills without external internet APIs.	In-memory Java parser evaluating 6 domain clusters (Backend, AI/ML, Frontend, DevOps, Mobile, Data).	*"If external AI APIs fail or hit rate limits, our local parser scores domain clusters in 15ms with zero downtime."*
Match to Ideal Roles	Scoring rubric connecting background to job titles.	45% weight on Projects, 30% on Stack, 15% on Seniority, 10% on Tooling with 3x project weighting.	*"We apply a 3x score multiplier for technologies proven in real project architectures over static skill list mentions."*

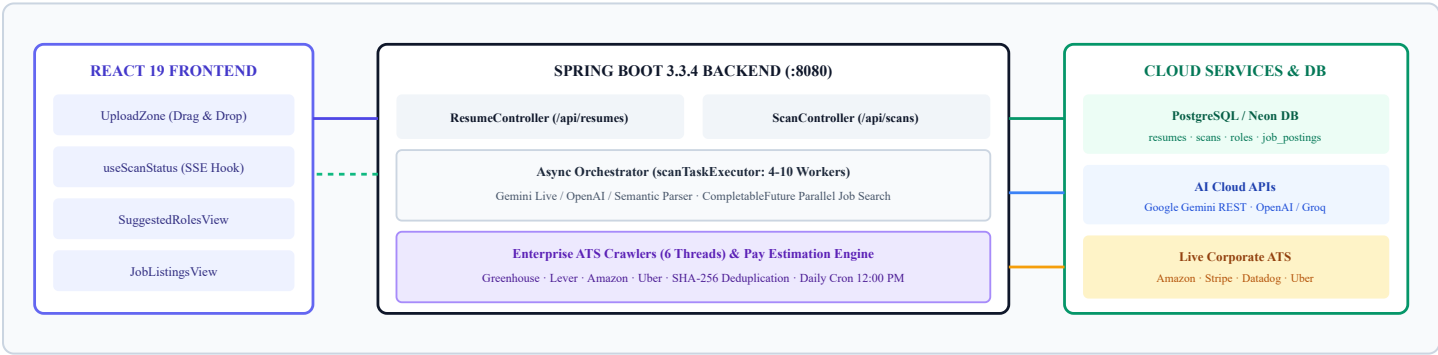
3. Resume Defense (Part 2) & System Architecture

📌 Resume Bullet Point 3:

> *"Engineered a secure, multi-threaded web crawler to aggregate live job postings from enterprise ATS systems, ensuring platform safety and data integrity through Apache Tika validation and SHA-256 deduplication."*

Term on Resume	Simple Meaning	Exact Code Implementation in Nous AI	What to Say in the Interview
Multi-Threaded Crawler	Background worker scraping multiple career sites simultaneously.	`CrawlOrchestratorService` on a 6-thread pool (`Executors.newFixedThreadPool(6)`) with 12s timeouts.	*"Crawls are parallelized across 6 worker threads with per-portal safety timeouts, preventing slow sites from blocking the batch."*
Enterprise ATS Systems	Applicant Tracking Systems used by Fortune 500 tech companies.	Adapters for Greenhouse API, Lever API, Amazon Jobs API, Uber API, and Jsoup Schema.org microdata.	*"We interface directly with public ATS endpoints (Greenhouse, Lever, Amazon) to guarantee 100% genuine apply links."*
Platform Safety	Protecting server stability, resources, and upload safety.	5MB size limit, Apache Tika byte inspection, and ClamAV TCP malware scanning.	*"Safety is maintained by verifying magic bytes before disk storage and scanning for malware over TCP socket."*
Data Integrity	Keeping database accurate, non-duplicate, and consistent.	Cascade deletes, foreign keys, unique constraints, and soft-expiration (`is_currently_open=false`).	*"Data integrity is enforced via relational constraints, composite hash deduplication, and automatic soft-expiration."*
Apache Tika Validation	Inspecting binary magic bytes in file headers.	Uses `Tika.detect()` on raw stream; rejects executables (`.exe`, `.sh`) disguised as `.pdf`.	*"Apache Tika reads file header magic bytes, preventing attackers from renaming malware executables to .pdf to bypass extension checks."*
SHA-256 Deduplication	Cryptographic 256-bit hashing to identify identical files/jobs.	Resumes: `SHA-256(fileBytes)` avoids duplicate storage. Jobs: `SHA-256(company:title:url)` prevents duplicate postings.	*"We use SHA-256 cryptographic digests for O(1) deduplication lookups, saving storage and preventing duplicate job entries."*

3.2 System Architecture Diagram



4. Deep Dive: Core System Pipelines

4.1 Security Ingestion & Text Extraction (Phase 1)

1. **MIME Header Sniffing:** `FileValidationService` passes raw file streams to **Apache Tika**. Tika inspects binary header bytes, rejecting renamed executables (e.g. `malware.exe` renamed to `resume.pdf`) with `InvalidFileException` (HTTP 400).
2. **SHA-256 Deduplication:** `ResumeService` computes a SHA-256 hex digest. If already in PostgreSQL, it reuses the existing `Resume` record and attaches a new `Scan`, eliminating redundant disk writes.
3. **ClamAV Virus Scanning:** If enabled, file bytes stream over a TCP socket to ClamAV daemon (`localhost:3310`) prior to disk persistence.
4. **Plain Text Extraction:** PDF text is extracted with Apache PDFBox `PDFTextStripper(sortByPosition=true)`. DOCX text is extracted via Apache POI `XWPFWordExtractor`.

4.2 AI Role Intelligence & 3-Tier Fallback (Phase 2)

Nous AI evaluates candidates using a **Staff Technical Recruiter** rubric:

45% Project Architecture Weight
Heavily scores actual microservices, ML pipelines, and databases built in the Projects and Experience sections.

30% Technical Stack Mastery
Evaluates depth of core programming languages (Java, Python, TypeScript) and modern frameworks (Spring Boot, React).

- **Tier 1 (Google Gemini Live REST):** Direct REST integration cascading across candidate models (`gemini-flash-latest` ➔ `gemini-2.5-flash` ➔ `gemini-2.0-flash`).
- **Tier 2 (OpenAI / Groq REST):** Calls `/chat/completions` with structured JSON schema output mode.
- **Tier 3 (Dynamic Semantic Parser):** In-memory Java parser evaluating 6 domain clusters with a **3x weight multiplier (+0.18 points)** for technologies verified inside the Projects section. Computes graduated confidence scores (0.65–0.96) in under 15ms.

4.3 Market Compensation Calibration Engine

`PayEstimationService` dynamically calibrates realistic salary bands across 4 vectors:

- **7 Geographic Regions & Currencies:** India (`₹ Lakhs/yr`), US/Remote (`$ USD/yr`), UK (`£ GBP/yr`), Europe (`€ EUR/yr`), Canada (`CAD $/yr`), Singapore (`SGD $/yr`), Australia (`AUD $/yr`).
- **7 Seniority Tiers:** Intern, Junior, Mid, Senior, Staff/Principal, Manager, Executive.
- **8 Domain Multipliers:** AI/ML (1.20x), Security (1.15x), Backend/Infra (1.10x), Product (1.05x), General SWE (1.00x), Design (0.95x), Sales (0.90x), Business Ops (0.80x).

4.4 Enterprise Portal Crawling & ATS Scraping (Phase 3)

- **Official ATS Adapters:** Connects to Greenhouse API, Lever API, Amazon Jobs search API, and Uber Careers.
- **Parallel Crawling Pool:** 6 worker threads with a 12-second per-portal safety timeout.
- **Deduplication & Soft Expiration:** Postings use composite `SHA-256(company_id:title:apply_url)`. Postings no longer listed during crawl runs are marked as `is_currently_open = false`.
- **Automated Daily Trigger:** Runs daily at **12:00 PM (Noon)** via Spring's `@Scheduled(cron = "0 0 12 * * *")`.

5. Database Schema & Complete REST API Catalog

5.1 Database Entity Relationship (ER) Model

Table Name	Primary Key	Key Columns & Constraints	Relationship & Cascade Behavior
`resumes`	`id UUID`	`file_hash (UK)`, `user_id`, `mime_type`, `extracted_text (TEXT)`	Root candidate record. Deletion cascades to scans and files.
`scans`	`id UUID`	`resume_id (FK)`, `status (ENUM)`, `created_at`, `completed_at`	Belongs to `resumes`. Cascade deletes roles & job listings.
`suggested_roles`	`id UUID`	`scan_id (FK)`, `role_title`, `rank_order`, `confidence_score`	Stores top 3 AI evaluated roles with verified skills CSV.
`job_listings`	`id UUID`	`scan_id (FK)`, `role_id (FK)`, `title`, `company`, `apply_url`	Matched job openings linked to specific target roles.
`companies`	`id UUID`	`name (UK)`, `adapter_type`, `adapter_config`, `is_active`	Monitored enterprise portals (Stripe, Amazon, Datadog, etc.).
`job_postings`	`id UUID`	`company_id (FK)`, `posting_hash (UK)`, `is_currently_open`	Scraped live enterprise openings with soft-expiration.
`crawl_runs`	`id UUID`	`started_at`, `completed_at`, `total_postings_found`	Daily crawl batch metrics and execution audits.
`crawl_results`	`id UUID`	`crawl_run_id (FK)`, `company_id (FK)`, `status`, `duration_ms`	Per-portal latency, error reason, and count metrics.

5.2 REST Endpoint Catalog & Status Codes

HTTP Method	Endpoint Path	Request Payload	Status & Response Details
`POST`	`/api/resumes`	`multipart/form-data` (file: PDF/DOCX, userId)	`202 Accepted`: Returns Resume metadata & active `scanId` (under 50ms).
`GET`	`/api/resumes/{id}`	None (Path Variable)	`200 OK`: Returns metadata, character count, extracted text preview.
`DELETE`	`/api/resumes/{id}`	None (Path Variable)	`204 No Content`: Cascades DB delete & removes physical file from disk.
`GET`	`/api/scans/{id}`	None (Path Variable)	`200 OK`: Returns scan status, top role title, and match confidence.
`GET`	`/api/scans/{id}/events`	None (Path Variable)	`text/event-stream` (SSE): Pushes real-time scan state changes.
`GET`	`/api/scans/{id}/roles`	None (Path Variable)	`200 OK`: Array of top 3 roles, scores, reasons, and skill arrays.
`GET`	`/api/scans/{id}/jobs`	None (Path Variable)	`200 OK`: Array of matched live job listings with apply URLs.
`GET`	`/api/crawler/companies`	None	`200 OK`: List of verified live monitored enterprise companies.
`POST`	`/api/crawler/trigger`	None	`200 OK`: Initiates async batch crawl across all active portals.
`GET`	`/api/crawler/postings`	`?query=java&limit=50`	`200 OK`: Live search over all crawled open enterprise postings.
`GET`	`/api/users/{id}/scans`	None (Path Variable)	`200 OK`: Full scan history across all resumes uploaded by user.

6. Master Technical Defense: Interview Questions & Answers

Q1: How does the asynchronous architecture prevent Tomcat request thread starvation?

"When a user uploads a resume via `POST /api/resumes`, we do not execute the long-running AI role evaluation within the HTTP request thread. The controller validates the file, computes the SHA-256 hash, extracts plain text, initializes a `Scan` entity with status `PENDING`, and dispatches the task to our custom `scanTaskExecutor` thread pool (Core: 4, Max: 10, Queue: 50) using `@Async`. The controller immediately returns `HTTP 202 Accepted` in under 50ms, freeing the Tomcat thread to accept new requests."

💡 Core Concept: Non-blocking asynchronous execution, ThreadPoolTaskExecutor, HTTP 202 Accepted.

Q2: Why did you choose Server-Sent Events (SSE) instead of WebSockets?

"WebSockets provide full-duplex bi-directional communication, which introduces unnecessary protocol complexity (binary framing, handshake overhead, ping/pong heartbeats) for a flow that only requires unidirectional status updates from server to client. SSE runs over standard HTTP/1.1 and HTTP/2 (`text/event-stream`), traverses firewalls seamlessly, and includes native browser reconnection via `EventSource`. We also engineered a 1500ms REST polling fallback in `useScanStatus.js` if a corporate proxy blocks event streams."

💡 Core Concept: Unidirectional event streaming, native reconnection, graceful polling degradation.

Q3: What is your 3-Tier AI Fallback Architecture and how does it guarantee 100% uptime?

"1. **Tier 1:** Google Gemini Live REST API with model cascading (`gemini-flash-latest` ➔ `2.5-flash` ➔ `2.0-flash` ➔ `1.5-flash`).
2. **Tier 2:** OpenAI / Groq `/chat/completions` endpoint with JSON schema output mode.
3. **Tier 3:** In-memory **Dynamic Semantic Parser** evaluating 6 domain clusters with 3x project weighting in under 15ms. Even if all external cloud AI providers are offline, the system never fails."

💡 Core Concept: Fault tolerance, circuit breaker fallback, in-memory zero-API parsing.

Q4: Why does Nous AI prioritize candidate projects (45%) over static skill lists (30%)?

"Traditional ATS systems fail because candidates stuff 50 buzzwords into their skills section. Our system prompts Gemini to act as a Staff Technical Recruiter, prioritizing concrete implementations (e.g., Spring Boot microservices, Kafka pipelines, React frontends) built in the Projects and Experience sections. Technologies verified in projects receive a 3x weight multiplier."

💡 Core Concept: Staff Recruiter evaluation rubric, 3x project section weighting multiplier.

Q5: How does the parallel job search execute across multiple roles during a scan?

"In `ScanService.java`, once the top 3 roles are determined, we construct a list of `CompletableFuture.supplyAsync()` tasks. Each role queries the job repository and pay calibration engine concurrently on separate worker threads. We use `CompletableFuture.allOf().join()` to aggregate all listings. This cuts job search latency from $3 imes 400 ext{ms} = 1.2 ext{s}$ down to under $400 ext{ms}$ total."

💡 Core Concept: Java Concurrency, CompletableFuture parallelization, sub-second aggregation.

Q6: How do you prevent malicious or fake file uploads?

"We enforce 3 layers of security: (1) **Size enforcement:** 5MB limit enforced at multipart and service layer. (2) **Magic-Byte Sniffing:** Apache Tika reads the true file header bytes, preventing executable files (e.g., `virus.exe`) renamed to `resume.pdf` from being processed. (3) **Anti-Malware:** File bytes are streamed over a TCP socket to a ClamAV daemon (`localhost:3310`) prior to disk persistence."

💡 Core Concept: Apache Tika magic-byte detection, ClamAV daemon TCP streaming, zero-trust file validation.

Q7: How does SHA-256 deduplication work in code?

"Upon receiving the file bytes, `ResumeService.java` calculates a SHA-256 hex digest (`MessageDigest.getInstance("SHA-256")`). We perform an indexed lookup in PostgreSQL on `resumes.file_hash`. If a match is found, we skip writing the file to disk and re-extracting text, immediately reusing the existing `Resume` record and creating a new `Scan` instance."

💡 Core Concept: Cryptographic hashing, idempotent storage, B-Tree index optimization.

7. Advanced Technical Defense & Architectural Trade-offs

Q8: How does the Pay Estimation Engine calculate compensation across different currencies?

"The engine uses a 4-tier matrix: (1) Geo-location detection maps the job to one of 7 regions (e.g., Bangalore ➔ India INR, London ➔ UK GBP). (2) Title analysis classifies seniority into 7 tiers (Intern to Executive). (3) Domain analysis applies multiplier weights (e.g., AI/Data is 1.20x, Distributed Systems is 1.10x). (4) The output is formatted in clean localized currency units (e.g., `₹38L - ₹65L / yr` for India or `$185,000 - $255,000 / yr` for the US)."

💡 Core Concept: Multi-factor compensation matrix, localized currency formatting, domain multipliers.

Q9: How does the crawler fetch real jobs from enterprise companies and handle closed jobs?

"We implemented modular career adapters: `GreenhouseAdapter` and `LeverAdapter` query official public REST APIs; `AmazonJobsAdapter` queries the official Amazon Jobs search API; `UberAdapter` queries internal career endpoints; and `GenericHtmlAdapter` parses Schema.org JSON-LD microdata using Jsoup. Each job posting is assigned a composite SHA-256 hash (`company_id:title:apply_url`). When a daily crawl completes, any posting not observed during the latest run is soft-expired by setting `is_currently_open = false`."

💡 Core Concept: Public ATS APIs, composite SHA-256 deduplication, soft-expiration lifecycle.

Q10: What database indexes exist and why?

"1. `resumes.file_hash`: Unique B-Tree index for $O(1)$ resume deduplication lookups.
2. `job_postings.posting_hash`: Unique B-Tree index for $O(1)$ crawler upserts and deduplication.
3. Foreign key indexes on `scans.resume_id`, `suggested_roles.scan_id`, and `job_listings.scan_id` for fast join and cascade operations."

💡 Core Concept: Database indexing, B-Tree index structure, foreign key constraints.

Q11: How did you configure HikariCP for serverless cloud databases (Neon DB)?

"Serverless containers have strict memory limits and connection pooling boundaries. In `application.properties`, we configured: `maximum-pool-size = 5`, `minimum-idle = 2`, `idle-timeout = 300000ms`, `max-lifetime = 900000ms`, and `prepareThreshold = 0` to prevent connection exhaustion."

💡 Core Concept: HikariCP pool tuning, Neon DB serverless, connection leak prevention.

Q12: How are exceptions mapped to HTTP responses across the backend?

"We implemented `GlobalExceptionHandler.java` using `@RestControllerAdvice`. All business exceptions (`InvalidFileException`, `ResumeNotFoundException`, `VirusScanException`, `LlmExtractionException`) are transformed into standardized RFC 7807 `ProblemDetail` JSON responses with appropriate HTTP status codes (400, 404, 422, 500)."

💡 Core Concept: RFC 7807 ProblemDetail, @RestControllerAdvice, uniform error handling.

Q13: What happens when a user deletes a resume? How is data privacy enforced?

"We enforce full Right-to-Erasure compliance. When `DELETE /api/resumes/{id}` is called, `ResumeService` and `ScanService` perform a transactional cascade delete: all child `job_listings`, `suggested_roles`, and `scans` records are removed from PostgreSQL, followed by the physical file being unlinked from disk storage."

💡 Core Concept: GDPR/Right-to-Erasure, transactional cascade deletion, physical file unlinking.

Final Defense Summary Statement

Nous AI represents an enterprise demonstration of full-stack software engineering. It combines zero-trust security ingestion, non-blocking asynchronous concurrency, multi-tier AI fallback reliability, direct corporate ATS integration, and real-time streaming UX into an end-to-end cloud platform. Every architectural choice—from Apache Tika byte inspection to SSE streaming and HikariCP connection tuning—was engineered for speed, fault-tolerance, and scale.